CGE-P LAB CURRICULUM · V2

Lab 4.4: Evidence Management & Chain of Custody (AWS)

You have a pipeline. You have an immutable vault. This lab connects them with cryptographic signing, so the evidence is provably yours, provably untampered, and provably timestamped. The auditor does not need to trust you. They verify.

Two things about verification scripts are worth knowing before you write one, and both are the kind that pass every happy-path test. Steps 3 and 4 cover them.

For reading. Code lines wrap to fit the page; copy code from docs/04_04_evidence_chain_of_custody.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/04_04_evidence_chain_of_custody.md` in your clone, not from the PDF.

Learning objectives

- Define chain of custody as four properties: **authenticity, integrity, timeliness, completeness**, and implement all four rather than three.
- Extend the Lab 4.3 pipeline with keyless Cosign signing via GitHub OIDC.
- Verify a bundle end-to-end, including a certificate identity check that actually constrains who signed it.
- Break the chain deliberately, four ways, and watch each break get caught.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- Lab 2.5 vault deployed. You have its bucket name.
- Lab 4.3 pipeline working on every PR.
- Cosign `>= 3.0` locally (`cosign version`).
- `jq` and `date` (GNU coreutils; macOS users see the Troubleshooting note).

Estimated time & cost

- 75 minutes.
- Free. Sigstore is free; marginal S3 cost.

Concept: why signing matters

Lab 2.5 made the bundle immutable. Object Lock prevents deletion, but it does not prove **who** created the bundle or **when**.

A determined insider with admin in your AWS account can stand up a different bucket, drop a tampered bundle in it, and point a sloppy auditor at that. Cosign closes the loop. The signature ties the bundle to a specific GitHub Actions run, on a specific repository, at a specific moment. The certificate Sigstore's Fulcio CA issues carries the OIDC subject, and Rekor timestamps it in a public transparency log.

None of that is bypassable by someone with admin in your AWS account, because none of it lives in your AWS account. That is the property you are buying.

Four ways the chain breaks. Close all four or you have a story, not a chain:

Property	Broken by	Closed by
Integrity	Byte-level tampering	SHA-256 recompute
Authenticity	Anyone can fabricate a bundle	Cosign signature + **strict cert identity**
Timeliness	Backdated evidence	Rekor transparency log entry
Completeness	Quietly dropping an inconvenient file	Manifest with a file list, checked

Most pipelines implement the first two and stop.

Architecture

```
PR opens
   |
   v
workflow run --+--> plan / policy / scan           (Lab 4.3)
               |
               +--> tar czf evidence bundle
               |
               +--> cosign sign-blob --bundle      (keyless, GitHub OIDC)
               |         |
               |         +--> Fulcio issues a short-lived cert
               |         |     carrying repo + workflow + ref as the subject
               |         +--> Rekor logs the signature with a timestamp
               |
               +--> aws s3 cp bundle + .sha256 + .sig.bundle + receipt.json
                         to s3://VAULT/runs/<run_id>/   (Object Lock applies)
               |
               v
auditor: scripts/verify-evidence.sh <run_id>
         |-- recompute SHA-256               integrity
         |-- cosign verify-blob, strict id   authenticity + timeliness
         |-- manifest file list matches      completeness
         +-- get-object-retention in future  preservation
         "CHAIN INTACT"
```

Step-by-step walkthrough

Step 1 Add Cosign to the workflow

```
      - name: Install Cosign
        uses: sigstore/cosign-installer@6f9f17788090df1f26f669e9d70d6ae9567deba6 # v4.1.2
        with:
          cosign-release: 'v3.1.3'
```

Then, after the evidence copy step:

```
      - name: Bundle, sign, and upload to vault
        id: sign
        if: always()
        env:
          VAULT:  ${{ vars.EVIDENCE_VAULT }}
          RUN_ID: ${{ github.run_id }}
          SHA:    ${{ github.sha }}
        run: |
          set -euo pipefail
          BUNDLE="evidence-${RUN_ID}-${SHA}.tar.gz"

          # Completeness: record what SHOULD be in the bundle before we build it.
          ( cd evidence && ls -1 > .manifest.txt && sha256sum * > .sha256sums.txt )

          ( cd evidence && tar czf "../${BUNDLE}" . )
          sha256sum "${BUNDLE}" | awk '{print $1}' > "${BUNDLE}.sha256"

          cosign sign-blob --yes --bundle "${BUNDLE}.sig.bundle" "${BUNDLE}"

          KEY_PREFIX="runs/${RUN_ID}"
          for f in "${BUNDLE}" "${BUNDLE}.sha256" "${BUNDLE}.sig.bundle"; do
            aws s3 cp "$f" "s3://${VAULT}/${KEY_PREFIX}/$f"
          done

          VERSION_ID=$(aws s3api head-object --bucket "${VAULT}" \
            --key "${KEY_PREFIX}/${BUNDLE}" --query VersionId --output text)

          jq -n \
            --arg run_id     "${RUN_ID}" \
            --arg vault      "${VAULT}" \
            --arg bundle_key "${KEY_PREFIX}/${BUNDLE}" \
            --arg version_id "${VERSION_ID}" \
            --arg sha256     "$(cat "${BUNDLE}.sha256")" \
            --arg commit     "${SHA}" \
            --arg repo       "${GITHUB_REPOSITORY}" \
            --arg workflow   "${GITHUB_WORKFLOW_REF}" \
            '$ARGS.named' > receipt.json

          aws s3 cp receipt.json "s3://${VAULT}/${KEY_PREFIX}/receipt.json"
```

```
      # The pass/fail decision moves to the LAST step, so evidence is signed
      # and stored even when the gate failed. A failed run is exactly the run
      # whose evidence you will want later.
      - name: Enforce gate result
        if: always()
        run: |
          test "${{ steps.conftest.outcome }}" = "success" || exit 1
          test "${{ steps.trivy.outcome }}"    = "success" || exit 1
```

Two things to notice. `jq -n '$ARGS.named'` builds the receipt instead of a bash heredoc, which means values containing quotes cannot corrupt the JSON. And the receipt now records `repo` and `workflow`, because Step 4's strict verification needs to know what identity to expect.

Step 2 Grant the role write access to the vault

The Lab 4.3 role has `ReadOnlyAccess` plus state access. Add a narrow vault write:

```
export AWS_PROFILE=cgep
VAULT=YOUR_VAULT_BUCKET
VAULT_KMS=YOUR_VAULT_CMK_ARN

aws iam put-role-policy --role-name cgep-grc-gate --policy-name vault-write \
  --policy-document "$(jq -n --arg v "$VAULT" --arg k "$VAULT_KMS" '{
    Version: "2012-10-17",
    Statement: [
      { Effect: "Allow",
        Action: ["s3:PutObject","s3:GetObject","s3:GetBucketLocation","s3:ListBucket"],
        Resource: ["arn:aws:s3:::\($v)","arn:aws:s3:::\($v)/*"] },
      { Effect: "Allow",
        Action: ["kms:GenerateDataKey","kms:Decrypt"],
        Resource: $k }
    ]}')"

gh variable set EVIDENCE_VAULT --body "$VAULT" --repo OWNER/REPO
```

Note the KMS statement. The Lab 2.5 vault is SSE-KMS with a customer-managed key, so `s3:PutObject` alone returns `AccessDenied` with a message about KMS that does not obviously say "add a KMS permission." An AES256 vault would not need it, which is why this is easy to miss.

No `s3:DeleteObject`. The pipeline writes evidence and must never be able to remove it. Object Lock would refuse anyway; not granting the permission means the attempt never reaches the bucket, and defense in depth is cheap here.

Step 3 The verify script

```
#!/usr/bin/env bash
# scripts/verify-evidence.sh <run_id> [--vault B] [--profile P] [--identity REGEX]
set -euo pipefail

RUN_ID="${1:?usage: verify-evidence.sh <run_id> [--vault B] [--profile P] [--identity REGEX]}"
shift || true

VAULT="${EVIDENCE_VAULT:-}"
PROFILE_ARG=""
IDENTITY="${COSIGN_IDENTITY:-}"

while [[ $# -gt 0 ]]; do
  case "$1" in
    --vault)    VAULT="$2";   shift 2 ;;
    --profile)  PROFILE_ARG="--profile $2"; shift 2 ;;
    --identity) IDENTITY="$2"; shift 2 ;;
    *) echo "Unknown arg: $1" >&2; exit 2 ;;
  esac
done

[[ -z "$VAULT" ]] && { echo "Set --vault or EVIDENCE_VAULT" >&2; exit 2; }

if command -v sha256sum >/dev/null 2>&1; then SHA256="sha256sum"
else SHA256="shasum -a 256"; fi

WORK=$(mktemp -d); trap 'rm -rf "$WORK"' EXIT; cd "$WORK"
PREFIX="runs/${RUN_ID}"

aws $PROFILE_ARG s3 cp "s3://${VAULT}/${PREFIX}/" . --recursive \
  --exclude "*" --include "evidence-*.tar.gz*" --include "receipt.json"

BUNDLE=$(ls evidence-*.tar.gz 2>/dev/null | head -1)
[[ -z "$BUNDLE" ]] && { echo "FAIL: no bundle found at ${PREFIX}" >&2; exit 1; }

echo "=== 1. Integrity (SHA-256) ==="
EXPECTED=$(cat "${BUNDLE}.sha256")
ACTUAL=$($SHA256 "${BUNDLE}" | awk '{print $1}')
[[ "$EXPECTED" == "$ACTUAL" ]] || { echo "FAIL: SHA mismatch"; exit 1; }
echo "  OK (${ACTUAL})"

echo "=== 2. Authenticity + timeliness (Cosign / Sigstore Rekor) ==="
# Derive the expected signer identity from the receipt unless overridden.
if [[ -z "$IDENTITY" && -f receipt.json ]]; then
  REPO=$(jq -r '.repo // empty' receipt.json)
  [[ -n "$REPO" ]] && IDENTITY="^https://github.com/${REPO}/\\.github/workflows/.*@refs/.*$"
fi
[[ -z "$IDENTITY" ]] && { echo "FAIL: no signer identity to check against." >&2; exit 1; }

cosign verify-blob \
  --bundle "${BUNDLE}.sig.bundle" \
  --certificate-identity-regexp "$IDENTITY" \
  --certificate-oidc-issuer 'https://token.actions.githubusercontent.com' \
  "${BUNDLE}"
```

```
echo "  OK (signed by an identity matching ${IDENTITY})"

echo "=== 3. Completeness (manifest) ==="
mkdir -p extracted && tar xzf "${BUNDLE}" -C extracted
if [[ -f extracted/.manifest.txt ]]; then
  MISSING=0
  while IFS= read -r f; do
    [[ -e "extracted/$f" ]] || { echo "  MISSING: $f"; MISSING=1; }
  done < extracted/.manifest.txt
  [[ $MISSING -eq 0 ]] || { echo "FAIL: bundle is missing files listed in its manifest"; exit 1; }
  ( cd extracted && $SHA256 -c .sha256sums.txt --quiet ) \
    || { echo "FAIL: a bundled file does not match its recorded hash"; exit 1; }
  echo "  OK ($(wc -l < extracted/.manifest.txt) files present and matching)"
else
  echo "FAIL: no manifest in bundle; completeness cannot be established"; exit 1
fi

echo "=== 4. Preservation (Object Lock retention) ==="
RETAIN_UNTIL=$(aws $PROFILE_ARG s3api get-object-retention \
  --bucket "${VAULT}" --key "${PREFIX}/${BUNDLE}" \
  --query 'Retention.RetainUntilDate' --output text)
# Compare as epoch seconds: correct by construction rather than by
# coincidence. See "the second finding" below for why the string test
# happens to work and why that is not a reason to keep it.
RETAIN_EPOCH=$(date -d "$RETAIN_UNTIL" +%s 2>/dev/null || date -jf "%Y-%m-%dT%H:%M:%S" "$
{RETAIN_UNTIL%.*}" +%s)
NOW_EPOCH=$(date -u +%s)
[[ "$RETAIN_EPOCH" -gt "$NOW_EPOCH" ]] || { echo "FAIL: retention expired"; exit 1; }
echo "  OK (retain until ${RETAIN_UNTIL})"

echo
echo "CHAIN INTACT for run ${RUN_ID}"
```

Step 4 The two defects this fixes

Worth stopping on, because both are the sort that never show up in a passing test.

The first: identity verification that verifies nothing. `--certificate-identity-regexp '.*'` is the path of least resistance, and it accepts a certificate issued to *any* GitHub Actions workflow in *any* repository on GitHub. Anyone with a public repo can produce a bundle that passes.

The signature was real. The question "whose signature" was never asked. Authenticity is not "a valid signature exists," it is "a valid signature from the identity I expect," and the difference is the entire control. v2 derives the expected identity from the receipt and refuses to run without one.

The second: a check that is right for the wrong reason. The obvious way to test retention is:

```
NOW=$(date -u +%Y-%m-%dT%H:%M:%SZ)
[[ "$RETAIN_UNTIL" > "$NOW" ]]
```

`RETAIN_UNTIL` comes back from AWS as `2026-04-27T18:30:33.696000+00:00`, `NOW` is `2026-04-27T18:30:33Z`, and `>` in bash is a lexicographic string comparison. Different formats compared as text looks like an obvious bug.

It is not. Generate two hundred thousand retention dates spread across a day either side of now and the string test agrees with an epoch comparison every single time. Both strings share a fixed-width, zero-padded ISO prefix, so comparing `YYYY-MM-DDTHH:MM:SS` as text *is* comparing it chronologically. The formats diverge only at the character after the seconds, and that character decides the outcome only when both sides land in the same second, where "not in the future" is the correct answer either way.

What does break it is a **non-UTC offset**:

Retention	Now	String test	Reality
`2026-08-18T16:00:00-05:00`	`2026-08-18T20:00:00Z`	expired	**an hour in the future**
`2026-08-19T00:00:00+05:00`	`2026-08-18T20:00:00Z`	valid	**an hour in the past**

AWS returns UTC for `get-object-retention`, so the string test is correct. It is correct **by coincidence**, resting on an upstream formatting choice nobody wrote down and nobody would notice changing.

Comparing epoch seconds is correct by construction, for any format that arrives. Prefer it, and understand that you are buying robustness rather than fixing a live defect.

There is a lesson here about verification work itself, and it cuts toward humility. "These strings have different formats, therefore the comparison is broken" is a plausible inference that survives review, sounds authoritative, and is wrong. It took a fuzz loop to find that out. **If you are going to claim a check is broken, make it fail in front of you first.** That is the same standard this lab applies to the controls; it applies to the criticism too.

Step 5 Break it four ways

```
EVIDENCE_VAULT=YOUR_VAULT bash scripts/verify-evidence.sh RUN_ID
```

Expect `CHAIN INTACT`. Now break each property in turn and confirm the right check catches it.

1. Integrity.

```
aws s3 cp "s3://${VAULT}/runs/${RUN_ID}/${BUNDLE}" /tmp/b.tar.gz
echo junk >> /tmp/b.tar.gz
$SHA256 /tmp/b.tar.gz   # differs from the .sha256 sidecar
```

Step 1 fails. And note you cannot write the tampered file *back*: Object Lock refuses to overwrite the existing key. The tampered copy exists only on your laptop.

2. Authenticity.

```
cosign sign-blob --yes --bundle /tmp/fake.sig.bundle /tmp/b.tar.gz
```

Sign it yourself with your own identity. Substitute that bundle and re-verify. Step 2 fails on certificate identity, because your personal OIDC subject is not the workflow subject in the receipt. **Under a `.*` identity regex this attack succeeds.**

3. Completeness.

```
tar xzf "${BUNDLE}" && rm plan.json && tar czf tampered.tar.gz .
```

Repackage without `plan.json` and re-sign it with the workflow identity, which an insider with repo write could do. Steps 1 and 2 pass. Step 3 fails, because `plan.json` is listed in `.manifest.txt` and is not there. Integrity and authenticity alone cannot detect this: the bundle is intact and correctly signed, it is simply missing the file that showed the finding.

4. Preservation. Wait for a GOVERNANCE 1-day retention to lapse, or point the script at an object in an unlocked bucket. Step 4 fails.

Four properties, four breaks, four catches. That table is the best page in your write-up.

Verification

- The vault holds `bundle.tar.gz`, `.sha256`, `.sig.bundle`, and `receipt.json` for at least one run.
- `verify-evidence.sh <run_id>` exits 0 with `CHAIN INTACT`.
- Each of the four tampering scenarios exits non-zero, at the expected step.
- A bundle signed by a different identity is rejected.

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-4-4"
mkdir -p "$EVIDENCE"
{
  echo "### unmodified bundle, expect CHAIN INTACT"
  bash scripts/verify-evidence.sh test-001 2>&1

  echo "### tampered bundle, expect failure"
  cp bundle.tar.gz /tmp/tampered.tar.gz && echo "x" >> /tmp/tampered.tar.gz
  sha256sum /tmp/tampered.tar.gz 2>&1
} | tee "$EVIDENCE/tamper-tests.txt"
```

Portfolio submission checklist

- [ ] `.github/workflows/grc-gate.yml` with Cosign install and the bundle/sign/upload step.
- [ ] `scripts/verify-evidence.sh` committed and executable.
- [ ] At least one run's full bundle in the vault.
- [ ] `evidence/lab-4-4/tamper-tests.txt` capturing all four failures.

- [ ] `WRITEUP.md` section mapping each chain property to the artifact that proves it, and naming which step catches which break.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`cosign sign-blob: failed to get OIDC token`.** The job needs `permissions: id-token: write`.
- **`cosign verify-blob` fails with certificate identity mismatch.** Working as designed if the bundle came from a different repo or workflow. If it is your own run, print the actual subject with `cosign verify-blob --bundle ... --certificate-identity-regexp '.*' ... 2>&1 | head` once, read it, then write the correct pattern. Do not leave `.*` in place.
- **`AccessDenied` on upload to the vault, mentioning KMS.** The role needs `kms:GenerateDataKey` on the vault CMK, not just `s3:PutObject`.
- **Rekor propagation race.** The public log can lag the signing call by about a second. CI naturally waits; this is a laptop-only race.
- **Object Lock rejects an overwrite.** Keys include `runs/<run_id>`, so each run is unique. A re-run that reuses a run ID gets a 403. Trigger a fresh run.
- **`date: illegal option -- d` on macOS.** BSD `date` differs from GNU. The script falls back to `date -jf`; or `brew install coreutils` and use `gdate`.
- **`sha256sum: command not found` on macOS.** The script detects and falls back to `shasum -a 256`.

Cleanup

Do not clean the vault. A 365-day-retention vault exists so evidence outlives the PR that produced it. For lab purposes Lab 2.5 deployed GOVERNANCE with 1-day retention, so bundles become deletable tomorrow. Production is COMPLIANCE, longer retention, no clean.

How this feeds the capstone

Every push now leaves a signed, timestamped, immutably-stored record of what was tested and what happened. An assessor who never meets you reconstructs it in minutes:

1. Read your OSCAL component (Lab 6.1).
2. Follow the evidence URI to a specific object version in the vault.
3. Run `verify-evidence.sh <run_id>`.
4. See `CHAIN INTACT`.

The capstone grader does exactly this and checks three things: the Cosign signature against the public Sigstore log, a SHA-256 recompute, and Object Lock retention. All three are in the script. The fourth check, completeness, is the one that will distinguish your submission, because most candidates will not have implemented it.

Revision history

v2 (current)

- Verification now constrains the signer identity, deriving it from the receipt and refusing to run without one. A permissive `.*` identity regex accepts a signature from any repository on GitHub.
- Added a completeness check: the bundle carries a manifest and per-file hashes, so removing a file from a correctly-signed bundle is detected.
- Retention compared as epoch seconds rather than as ISO strings, for robustness against non-UTC offsets.
- The pass/fail decision moved to the last workflow step, so evidence is signed and stored even when the gate fails.
- The receipt is built with `jq -n` rather than a heredoc, and records the repository and workflow so verification knows what identity to expect.
- The vault write policy gained `kms:GenerateDataKey`, needed now the vault uses a customer-managed key.

v1

Initial release: three of the four chain properties, with authenticity checked against a permissive identity regex and no completeness check.